Functional or Black box Testing

If we do not have access to the code, we can test the s/w only by executing the s/w giving input and observe if the output . In this scenario, choosing the right kind of input is critical towards find the hidden bugs .

Black Box Testing

- In the **black-box testing (functional testing)**, **test cases are designed** from an examination of the **input/output values only** and **no knowledge of design, or code is required**.
- **A program P** is viewed as a function transforming input to output. Given a set of inputs x_i, P computes y_i such that $\boldsymbol{y_i = P(x_i)}$

Important points on Black box testing

- We need to Precisely **identify the domain of each input and output variable**.
- From the **domain** of each **data** variable**,** we need to choose **few selected values** having important properties.
- May be we need to consider **combinations of special values** from **different input domains** to design test cases.
- We need to consider input values such that the program under test produces special values from the domains of the output variables.

Types of Black Box Testing

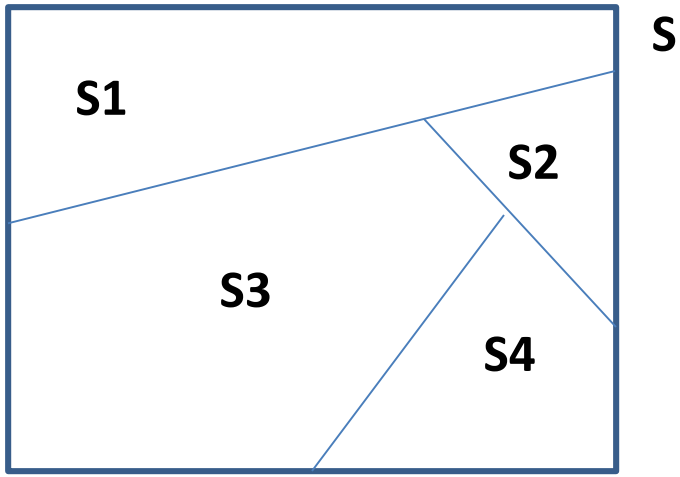
- Equivalence Class Partitioning or Input Space Partitioning
- Boundary Value Analysis
- Decision Tables
- Random Testing
- Pair-Wise Testing: Orthogonal arrays

Equivalence Class Partitioning

- Equivalence partitioning or **Equivalence Class Partitioning** (ECP) is a software testing technique that **divides the input data** of a software unit **into partitions of equivalence class** from which test cases can be derived.
- This **partitioning is done** such that the **behavior of the program is similar** for **every input** data **belonging to the same equivalence class**.

Partition of a Set

- Given a set S, a partition of S is a set $\{S_1, S_2, ---, S_n\}$ of sub sets of S such that
  - The subsets S_i of S are pair wise disjoint, i.e. S_i intersection S_j=Empty set
  - The union of subsets S_i is the entire set S
- For example **set of integer numbers** may be divided in to **3 sub sets**, **-ve** , **0** and **+ve**.



Equivalence Class Partitioning

- The main idea behind defining the equivalence classes is that **testing the code with any one value belonging to an equivalence class is as good as testing the software with any other value belonging to that equivalence class**.
- The following are **some general guidelines** for **designing the equivalence classes**:
  - 1. **If** the **input data values** to a system can be **specified by a range of values, then one valid and two invalid equivalence classes should be defined**.
  - 2. **If the input data assumes values from a set of discrete members of some domain**, then **one equivalence class for valid input** values and **another equivalence class for invalid input** values should be defined.

Equivalence Partitioning Examples

A Date field in an user interface accepts values between 1 and 31.

Invalid Partition	Valid Partition	Invalid Partition
0	1	32
-1	2	33
-2	3	.
.	.	.
.	31	.

A office visit timing field in an user interface accepts values between 9.15 and 17.40, except between 13.00 and 13.59 (lunch break).

Invalid Partition	Valid Partition	Invalid Partition	Valid Partition	Invalid Partition
9.14	9.15	13.0	14.0	17.41
9.13	9.16	13.1	14.1	17.42
.	.	.	.	.
.	12.59	13.59	17.40	.

Equivalence Class Partitioning: Example

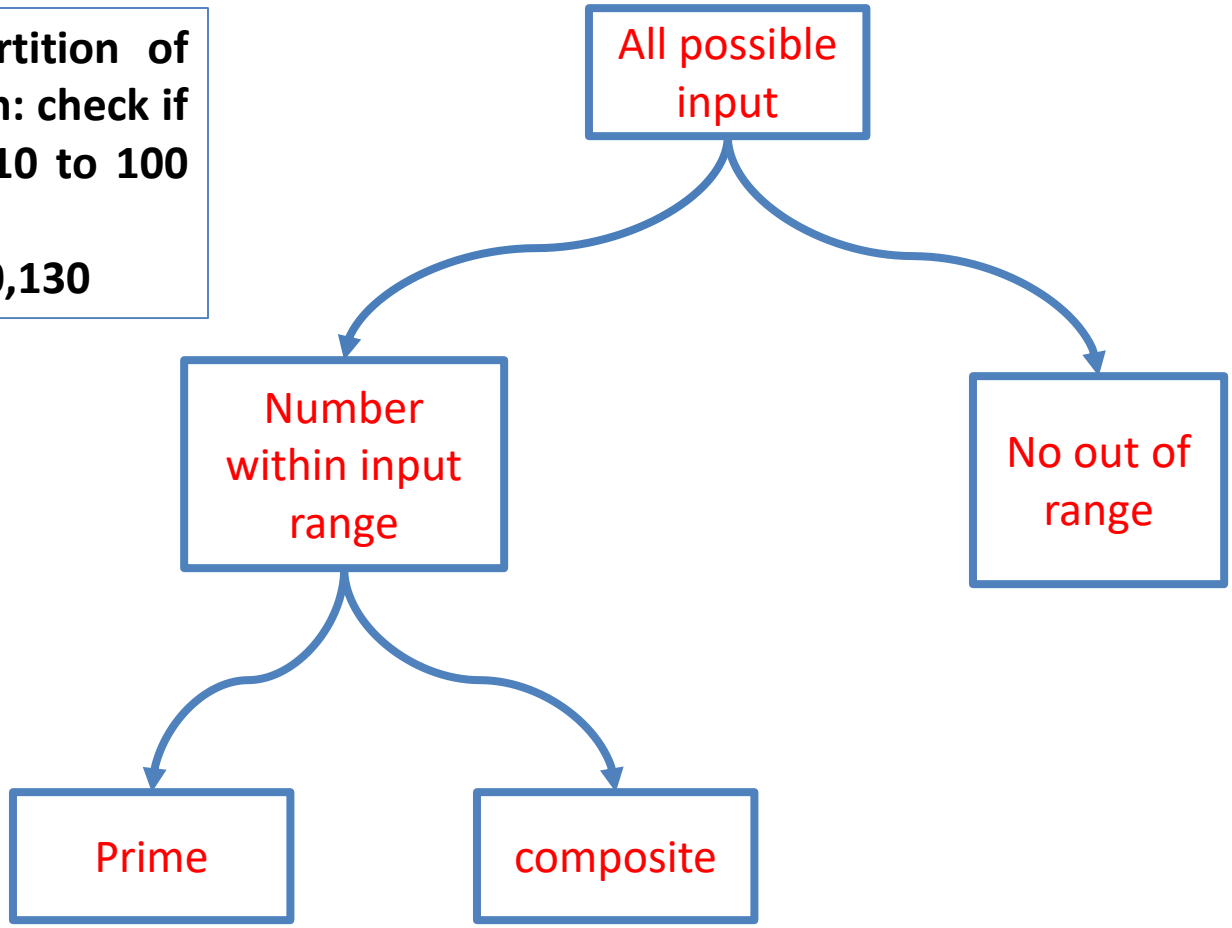
- Consider a s/w system that **calculates Income tax** based on rule:
- Gross Income
  - Between Rs. 1 and Rs. 2,50,000: Tax to be paid: Nil
  - Between Rs. 2,50,001 and 5,00,000: Tax to be paid: 10% of the income beyond 2,50,000
  - Between Rs. 5,50,001 and 10,00,000: Tax to be paid: 20% of the income beyond 2,50,000
  - More than 10,00,001: Tax to be paid: 30% of the income beyond 2,50,000
- **We need to have five test cases, one invalid input (GI<1)** and **four more input for the four ranges**

Equivalence Partitioning Example

- Design the black-box test suite for the following program.
  - The program computes the **intersection point of two straight lines and displays the result**. It reads **two integer pairs** (**m1, c1**) and (**m2, c2**) defining the two straight lines of the form **y=mx + c**.
- The equivalence classes are the following:
  - **Parallel lines (m1=m2, c1≠c2)**
  - **Intersecting lines (m1≠m2)**
  - **Coincident lines (m1=m2, c1=c2)**
- Now, selecting one representative value from each equivalence class, the test suit (2, 2) (2, 5), (5, 5) (7, 7), (10, 5) (10, 5) are obtained.

Equivalence Partitioning Examples

Find out equivalent partition of the input for the program: check if a number within range 10 to 100 is prime or not.
Possible test case:9,17,50,130



Boundary Value Analysis

- A type of **programming error** **frequently occurs at the boundaries of different equivalence classes of inputs**.
- The reason behind such errors might purely be due to psychological factors.
- Programmers often fail to see the special processing required by the input values that lie at the boundary of the different equivalence classes.
- For example, **programmers may improperly use < instead of <=, or conversely <= for <**.
- Boundary value analysis leads to selection of test cases at the boundaries of the different equivalence classes.

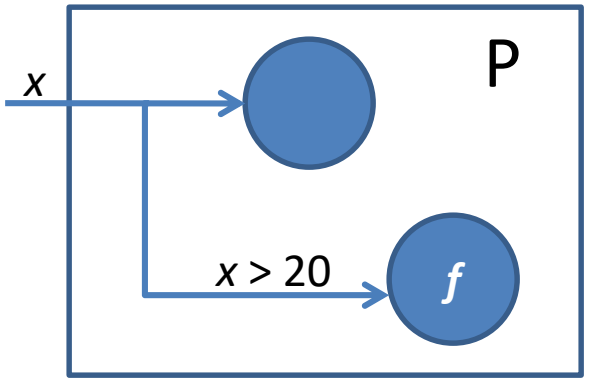
Boundary Value Analysis Example

A Date field in an user interface accepts values between 1 and 31.

Invalid Partition – valid partition lower boundary		Invalid Partition – valid partition upper boundary	
Boundary value just below boundary	**Boundary value just above boundary**	**Boundary value just below boundary**	Boundary value just above boundary
0	**1**	**31**	**32**

Testing in a specific context

- Though functional testing is a black box testing, **sometimes minimal context information is necessary** for getting relevant values of i/p and o/p.
- Example: suppose for an input variable x there is a predicate x>=20 that we are unaware of.
- We are likely to test input values for x=+k, x=-k and x=0
- The function will be invoked just for once, x=+k
- The valid inputs for x w.r.t. testing f are x<20, x>20 and x=20.



Decision Table

- Equivalence partitioning consider each input separately. We can not combine conditions.
- Decision table handles multiple inputs by considering different combinations of equivalent classes.
- It is a popular testing criteria for several categories of s/w.

Decision Table Example

		Rule Combinations							
Conditions	**Values**	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
C1	**Y, N, -**	**Y**	**Y**	**Y**	**Y**	**N**	**N**	**N**	**N**
C2	**Y, N, -**	**Y**	**Y**	**N**	**N**	**Y**	**Y**	**N**	**N**
C3	**Y, N, -**	**Y**	**N**	**Y**	**N**	**Y**	**N**	**Y**	**N**
Effects									
E1		**1**		**2**	**1**				
E2			**2**	**1**			**2**	**1**	
E3		**2**	**1**	**3**		**1**	**1**		
Checksum	**8**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**

Note: If don't care is also considered, total no. of rules will be 3^3= 27

Decision Table Example

- A decision Table contains:
  - A set of conditions and a set of effects arranged in a column
  - Each condition has three possible values: Yes(Y), No(N) and Don't care(-) in the second column
  - For each combination of the three conditions, there are a set of rules from R1 to R8. Each rule has a Y/N/- response and contains an associated set of effects (E1, E2 and E3).
  - For each effect, an effect sequence number specifies the order in which the effect should be carried out if the associated set of conditions are satisfied. For e.g. if C1 and C2 are T, but no C3, then E2 should be followed by E3.

Decision Table Example

Conditions	R1	R2	R3
Withdrawal Amount <=Balance?	Y	Y	N
Credit Granted?	-	Y	N
Effects			
Withdrawal Granted	Y	Y	N

Random Testing

- In random testing, **test inputs are selected randomly from the input domain**.
- Example:
  - A function z=sqrt(x) accept values between 1 to 10^8 for x and find square root of x up to accuracy within 2×10^{-4}
  - To test this function, one can generate uniformly distributed pseudorandom numbers for x and obtain outputs z.
  - For each value of x, we can compute z^2 and compare back with x. The error margin should be well within 2×10^{-4}. Otherwise program must be rectified.